

## LLM Fundamentals

Mengenal Large Language Models (LLM) berbasis Transformer: dari cara kerjanya (token → embedding → attention → prediksi kata) sampai cara memakai, memproduksi, meng-adaptasi, mengukur, spesialisasi, dan deploy ke edge. Delapan notebook (BANGUN → PAKAI → PRODUKSIKAN → ADAPT → UKUR → SPESIALISASI → DEPLOY), semua jalan di Colab Tesla T4 16GB gratis.

Alur: Teks/Prompt → Tokenization (token -> ID) → Embedding (ID -> vektor) → Transformer block + Attention berlapis → Prediksi token berikutnya → Sampling (temperature) → Teks output (autoregressive, ulang per token)

## NLP vs LLM

- NLP = bidang luas pemrosesan bahasa
- LLM = Large Language Model, satu jenis model NLP
- Dilatih pada teks masif (miliaran kata) + arsitektur transformer
- 1 LLM mengerjakan banyak tugas tanpa dilatih ulang per tugas

► memahami posisi LLM di dalam NLP

## Skala &amp; Evolusi

- Evolusi: n-gram -> RNN/LSTM -> Transformer (2017)
- Transformer proses kata paralel via attention, bukan satu per satu (RNN)
- Makin banyak parameter -> makin mampu (sampai titik tertentu)
- Data, arsitektur & fine-tuning sama pentingnya; BERT 110M -> GPT-3 175B

## Tokenization

- Model baca token, bukan huruf/kata utuh
- Token sering subword (potongan kata); tiap token -> ID angka
- Kata umum = 1 token; kata jarang dipecah jadi beberapa
- Semua teks diubah jadi token ID dulu

► langkah pertama mengubah teks jadi angka

## Bagaimana Transformer Bekerja

- Bayangkan rapat: tiap kata adalah satu peserta yang dengar semua peserta lain
- Langkah: teks -> token -> embedding (vektor) -> attention berlapis -> tebak token berikutnya
- Tiap lapisan = ronde diskusi: tiap kata memperbarui "pemahamannya" dari kata sekitar
- Setelah banyak ronde, model menebak kata paling masuk akal sebagai lanjutan

► intuisi inti cara kerja model

## Embedding = Cara "Memahami"

- Tiap kata diubah jadi daftar angka (embedding/vektor)
- Makna mirip -> vektor mirip ("raja" dekat "ratu", jauh dari "meja")
- "Memahami" = mengenali POLA dari teks masif + memakai KONTEKS lewat attention
- Ini statistik/pola, BUKAN kesadaran atau pikiran seperti manusia

► demistifikasi: kenapa model seolah paham bahasa

## Attention: Q, K, V, O

- Analogi diskusi kelompok untuk tiap kata:
- Q** (Query) = pertanyaan kata: "kata mana yang relevan untukku?"
- K** (Key) = name-tag/label keahlian tiap kata
- V** (Value) = isi informasi tiap kata
- Attention = cocokkan Q-ku ke semua K, ambil campuran-berbobot dari V
- O** (Output proj.) = merangkum kembali info yang terkumpul

```
skor = Q · KT / √d
bobot = softmax(skor)
hasil = bobot · V # lalu lewat O
```

► jantung transformer, dijelaskan tanpa rumus berat

## Generation &amp; Sampling

- LLM autoregressive: prediksi 1 token lalu ulang
- Token baru jadi input token selanjutnya
- temperature rendah = fokus; tinggi = kreatif/acak
- top\_p & repetition\_penalty bantu hasil rapi & tidak mengulang

```
model.generate(**inputs, do_sample=True,
               temperature=0.7, top_p=0.9,
               repetition_penalty=1.1, max_new_tokens=200)
```

► atur keseimbangan fokus vs kreativitas output

## ① Transformer dari Nol (BANGUN)

- PyTorch murni; char tokenizer (per huruf)
- Rakit self-attention: Q·K<sup>T</sup>/√d -> softmax -> jumlah-berbobot V
- Causal mask (tak boleh lihat masa depan) + multi-head + LayerNorm/GELU/residual
- Susun TinyGPT (~0.8M params), latih di korpus Indonesia kecil, lalu generate

```
scores = Q @ K.transpose(-2,-1) / d**0.5
scores = scores.masked_fill(mask==0, -inf)
out = softmax(scores) @ V
```

► paham transformer dengan membangunnya sendiri

## ② Pakai Model Instruct (PAKAI)

- Qwen2.5-3B-Instruct (fp16); model siap-jawab
- Chat template + system prompt (apply\_chat\_template, format ChatML)
- Prompt engineering: zero-shot & few-shot (beri contoh di prompt)
- Klasifikasi: bert-tiny (head mentah -> LABEL\_0/1 ~0.5) vs distilbert-sst2 (terlatih) + zero-shot bart-large-mnli (label bebas); chatbot Gradio

```
msgs=[{"role":"system","content":"..."},
       {"role":"user","content":"Halo"}]
inputs=tok.apply_chat_template(msgs,
                                add_generation_prompt=True, return_dict=True,
                                return_tensors="pt").to(model.device)
model.generate(**inputs)
```

► cara benar memakai LLM chat siap pakai

## ③ Quantization 4-bit (PRODUKSIKAN)

- Mistral-7B fp16 ~16GB -> OOM (tak muat) di T4
- Quantization 4-bit NF4: kecilkan bobot -> muat ~5.5GB
- Di T4 (compute 7.5) WAJIB compute\_dtype float16; bf16 CRASH
- Profil memori GPU; batching (padding\_side='left'); streaming token (TextStreamer)

```
bnb = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16) # bf16 crash!
AutoModelForCausalLM.from_pretrained(
    name, quantization_config=bnb, device_map="auto")
```

► jalankan model 7B di GPU gratis 16GB

## ④ Fine-Tuning LoRA/QLoRA (ADAPT)

- Model raksasa = miliaran "kenop" (weights); latih ulang semua terlalu mahal
- LoRA**: BEKUKAN model besar, tempel matriks kecil (low-rank) yang dilatih — analogi: jangan tulis ulang buku, cukup tempel sticky-note/errata; hanya ~1.2% params dilatih
- QLoRA**: kecilkan dulu ke 4-bit (muat di T4), BARU tempel sticky-note LoRA — analogi: buku edisi saku 4-bit + sticky-note
- Demo: Qwen2.5-1.5B + Trainer, ajari fakta Navasena, ~2 menit; banding base vs hasil via disable\_adapter()

```
LoraConfig(r=8, lora_alpha=16,
           target_modules=["q_proj","v_proj"])
# base 4-bit dibekukan, hanya adapter dilatih
```

► ajari model baru tanpa GPU mahal

## ⑤ Evaluasi (UKUR)

- Metrik teks: ROUGE/BLEU (cocok kata), BERTScore (cocok makna), perplexity
- Uji statistik: paired t-test, Cohen's d, 95% CI
- LLM-as-judge: minta LLM lain menilai (Qwen2.5-3B lokal / cloud NVIDIA NIM)
- Pelajaran: metrik bisa unggulkan model lebih besar, tapi p>0.05 = belum signifikan secara statistik

```
t, p = ttest_rel(skor_A, skor_B)
# p < 0.05 -> beda benar-benar nyata
```

► buktikan model mana lebih baik secara jujur

## ⑥ SLM Specialist (SPESIALISASI)

- SLM bukan model bodoh — "sempit tapi dalam", fokus satu tugas
- Fine-tune Qwen3-0.6B (QLoRA) jadi specialist ekstraksi order → JSON
- Head-to-head: base 0.6B 78% → specialist 95% > generalist SmoLLM3-3B 72% (5× lebih kecil tetap menang)
- Kemenangan paling andal = memory (3,7× lebih hemat VRAM); throughput di T4 ~sama (jujur, bukan selalu lebih cepat)
- Distillation: model kecil belajar dari output model raksasa → SLM modern bisa lampau LLM lama

```
LoraConfig(r=8, target_modules=
["q_proj","v_proj"])
tok.apply_chat_template(msgs,
                        enable_thinking=False) # task format
```

► kapan pakai model kecil yang di-fine-tune

## ⑦ Task Gallery (GALLERY)

- Satu resep QLoRA dipakai-ulang untuk banyak tugas
- 3 specialist dari resep sama — intent classification, sentiment-aspect, domain FAQ
- Semua lompat ke 100% (base→specialist: 57→100, 0→100, 12→100)
- Yang berubah hanya dataset kecil tiap tugas
- Pakai gc.collect() untuk bebaskan CPU-RAM saat memuat banyak model berurutan

```
# satu fungsi, banyak specialist
run_task("intent", data, base="Qwen/Qwen3-0.6B")
```

► sekali kuasai resep, cetak banyak specialist

## ⑧ Deployment ke Edge (DEPLOY)

- Merge adapter ke base fp16 (BUKAN base 4-bit → NaN → output "GGGG")
- Convert ke GGUF lalu quantize — Q8\_0 ~0,6 GB (near-lossless) / Q4\_K\_M ~0,4 GB
- Push ke HuggingFace → device tinggal ollama create (nol scp)
- Modelfile WAJIB TEMPLATE ChatML + SYSTEM prompt (kalau telanjang, system prompt hilang)
- Client OpenAI-compatible → pindah Ollama/vLLM/NIM cukup ganti base\_url
- Edge = Ollama/llama.cpp/TensorRT, datacenter = vLLM/NIM

```
model.merge_and_unload() # dari base fp16!
llama-quantize model.gguf out.gguf Q8_0
ollama create spesialis -f Modelfile
```

► kirim specialist ke Mac / Jetson edge

Quick-Ref Library & Model (8 Notebook)

| Notebook                           | Library/Model                                                        | Tujuan                                                                |
|------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------|
| 00 · BANGUN — Transformer dari Nol | PyTorch (TinyGPT ~0.8M)                                              | Rakit self-attention & latih GPT mini sendiri                         |
| 01 · PAKAI — Model Instruct        | Qwen2.5-3B-Instruct (fp16); distilbert-sst2; bart-large-mnli; Gradio | Chat template, prompt eng, klasifikasi, chatbot                       |
| 02 · PRODUKSIKAN — Quantization    | Mistral-7B-Instruct-v0.3 + BitsAndBytes 4-bit NF4                    | Muat 7B ~5.5GB di T4, batching & streaming                            |
| 03 · ADAPT — Fine-Tuning           | QLoRA: Qwen2.5-1.5B + LoRA adapter + Trainer                         | Latih ~1.2% params, ajari fakta baru ~2 menit                         |
| 04 · UKUR — Evaluasi               | ROUGE/BLEU/BERTScore, perplexity, NVIDIA NIM judge                   | Metrik + uji statistik + LLM-as-judge                                 |
| 05 · SPESIALISASI — SLM Specialist | Qwen3-0.6B (QLoRA) vs SmoLLM3-3B                                     | Specialist kecil kalahkan generalist (95% vs 72%), 3,7× hemat memory  |
| 06 · GALLERY — Task Gallery        | Qwen3-0.6B (QLoRA), run_task() reusable                              | Satu resep → banyak specialist (intent/sentiment/FAQ, semua 100%)     |
| 07 · DEPLOY — Edge Deployment      | GGUF + Ollama (Mac/Jetson), OpenAI client                            | Merge fp16 → GGUF → quantize → Ollama; pindah target = ganti base_url |

Istilah Kunci

**LLM** — Large Language Model: model NLP raksasa berbasis transformer, dilatih pada teks masif.

**Token** — Potongan teks (sering subword) yang dipetakan ke ID angka untuk dibaca model.

**Embedding** — Vektor angka mewakili token; kata mirip makna -> vektor mirip.

**Attention** — Mekanisme model menimbang kata mana yang paling relevan saat memproses kalimat.

**Q / K / V** — Query (pertanyaan kata), Key (label keahlian kata), Value (isi info kata); attention mencocokkan Q ke semua K lalu ambil campuran-berbobot V.

**Transformer** — Arsitektur (2017) yang memproses kata paralel via attention, dasar LLM modern.

**Autoregressive** — Cara generasi: prediksi 1 token, jadikan input berikutnya, ulang terus.

**Temperature** — Parameter sampling: rendah = fokus/deterministik, tinggi = kreatif/acak.

**Chat template** — Format pesan baku (system/user/assistant, mis. ChatML) lewat apply\_chat\_template agar model instruct paham peran.

**Quantization** — Kecilkan presisi bobot (mis. 16-bit -> 4-bit NF4) agar model muat di GPU kecil & lebih hemat memori.

**LoRA** — Bekukan model besar, tambah matriks kecil (low-rank) yang dilatih (~1%); analogi sticky-note, bukan tulis ulang buku.

**QLoRA** — LoRA di atas model 4-bit: kecilkan dulu (quantization) lalu tempel adapter; muat di T4 gratis.

**Perplexity** — Ukuran "kebingungan" model menebak teks; makin rendah makin baik.

**BLEU / ROUGE** — Metrik kemiripan kata output vs jawaban acuan (BLEU presisi, ROUGE recall); BERTScore menilai kemiripan makna.

**SLM** — Small Language Model: model bahasa kecil (mis. 0.6B~3B); efisien & bisa jadi specialist hebat di satu tugas.

**Specialist vs Generalist** — Specialist di-fine-tune fokus satu tugas (bisa kalahkan model umum lebih besar di tugas itu); generalist serba-bisa tapi tak sedalam itu pada tugas sempit.

**Distillation** — Melatih model kecil meniru output model raksasa, sehingga SLM modern bisa melampaui LLM lama.

**GGUF** — Format file model universal untuk Ollama & llama.cpp; dipakai untuk deployment edge.

**Merge (adapter)** — Gabungkan adapter LoRA ke base fp16 (BUKAN 4-bit) sebelum export GGUF; merge ke 4-bit menyuntik NaN → output rusak.

**Ollama** — Runtime ringan untuk menjalankan GGUF di edge (Mac/Jetson); OpenAI-compatible.

**OpenAI-compatible API** — Antarmuka standar; pindah dari Ollama ke vLLM/NIM cukup ganti base\_url.